

Relazione progetto AWC 2025/2026

SOCIAL NETWORK FOR MUSIC SN4M

Progetto Full

Alessandro Lucchesi

Applicazioni Web e Cloud - A.A. 2025/2026

1. Introduzione

SN4M (Social Network for Music) è una web application dedicata alla gestione di playlist musicali e alla creazione di comunità di utenti con interessi musicali comuni. L'applicazione implementa i tre macro-scenari richiesti dalla traccia del progetto: gestione delle playlist, gestione delle comunità musicali e gestione delle condivisioni.

Dopo la registrazione, l'utente può accedere al sito, modificare i propri dati e le proprie preferenze musicali, cercare brani e artisti tramite le API REST di Spotify, creare playlist private, unirsi a comunità e rendere pubbliche le proprie playlist all'interno dei gruppi di cui fa parte. È inoltre prevista l'importazione nel proprio profilo di playlist condivise da altri utenti.

Modalità di progetto: Il progetto è stato sviluppato come Progetto Full individuale, utilizzando HTML5, CSS3 e JavaScript lato client, con persistenza dei dati tramite Web Storage del browser.

2. Struttura e design

L'applicazione utilizza una struttura a pagina singola: le principali aree funzionali sono definite come sezioni HTML e vengono mostrate o nascoste dinamicamente tramite JavaScript. In questo modo la navigazione tra Home, ricerca, comunità, dettaglio delle playlist e autenticazione avviene senza ricaricare la pagina.

- HTML5 per la struttura dell'interfaccia, dei form, delle viste, delle tabelle e dei modali.
- CSS3 per tema, colori del brand, effetti hover, badge, overlay e personalizzazioni.
- Bootstrap 5.3 per griglia responsive, modali, dropdown, pulsanti e componenti.
- Bootstrap Icons per le icone di navigazione, profilo, playlist, community e player.
- JavaScript per autenticazione, Spotify, ricerca, playlist e comunità.
- localStorage per la persistenza di utenti, sessione, playlist, comunità e condivisioni.

3. Organizzazione dei file

File	Compito
index.html	Contiene la struttura principale: sidebar, header, Home, autenticazione, vista playlist, bacheca community, overlay di ricerca, player e modali.
style.css	Gestisce tema chiaro/scuro, colore del brand, card, hover, dropdown, badge, overlay e scrollbar.
SpotifyAPI.js	Gestisce token Spotify, categorie musicali e ricerca di brani e artisti.
search.js	Gestisce la ricerca globale live dei brani e la stampa dei risultati nel DOM.
user.js	Gestisce utenti, sessione, registrazione/login, modifica profilo, preferenze musicali e localStorage.
playlist.js	Gestisce playlist private, modifica del nome, aggiunta/rimozione brani, tag, dettaglio, eliminazione e condivisione.

community.js	Gestisce community, iscrizione, modifica/cancellazione, membri, bacheca, ricerca delle playlist condivise e importazione.
index.js	Gestisce viste, navigazione, controllo accessi, cambio tema e player grafico.

4. Usabilità

- Interazioni: pulsanti con hover, ombre e variazioni visive per indicare le azioni disponibili.
- Feedback: alert e confirm informano l'utente sull'esito delle operazioni o richiedono conferma prima di cancellazioni irreversibili.
- Gestione dati vuoti: messaggi dedicati per assenza di playlist, comunità, risultati di ricerca o brani.
- Tag e preferenze: generi e tag sono rappresentati tramite badge e possono essere rimossi direttamente dall'interfaccia.
- Azioni protette: se un utente non autenticato tenta di creare playlist o utilizzare funzioni riservate viene portato alla schermata di accesso.

5. Layout, navigazione e responsive design

Il layout è composto da una sidebar laterale, un'area principale con viste dinamiche e un footer fisso che rappresenta il player. La sidebar permette di accedere a Home, Cerca e Community e mostra inoltre l'elenco delle playlist personali.

- Da dimensione md in poi la sidebar è visibile; sui dispositivi più stretti viene nascosta.
- Le griglie di playlist e risultati utilizzano colonne Bootstrap che cambiano numero in base alla larghezza dello schermo.
- I modali utilizzano larghezze responsive e il form di creazione playlist passa da due colonne a disposizione verticale su schermi più stretti.
- Il tema chiaro/scuro viene applicato tramite l'attributo data-bs-theme e memorizzato nel localStorage come theme-override.
- Il player grafico resta visibile nel footer e mostra titolo, artista e copertina del brano selezionato.

6. Dettagli di implementazione

6.1 Autenticazione utenti e gestione profilo

La registrazione raccoglie nome utente, email, password, generi preferiti e artisti preferiti. I generi vengono recuperati da Spotify e selezionati tramite checkbox, mentre gli artisti vengono cercati in modo dinamico. Prima della creazione dell'account viene controllato che l'email non sia già presente.

Il login confronta email e password con gli utenti memorizzati. Quando l'accesso ha successo l'oggetto dell'utente viene salvato in `sn4m_currentUser` e l'interfaccia viene aggiornata mostrando playlist, comunità e pulsante profilo. Il profilo consente di modificare username, email, generi e artisti preferiti, effettuare logout o eliminare definitivamente l'account.

6.2 Integrazione con Spotify Web API

Il modulo `SpotifyAPI.js` incapsula le chiamate REST verso Spotify. Viene usato il flusso Client Credentials per ottenere un access token, mantenuto in cache fino a poco prima della scadenza. Il token viene poi inserito nell'header Authorization delle richieste successive.

- `getCategories()`: recupera le categorie musicali utilizzate nei menu di generi e tag.
- `searchTrack()`: cerca fino a 20 brani e restituisce id, titolo, artisti, anno, durata e immagine.
- `searchArtist()`: cerca fino a 5 artisti e restituisce id, nome e immagine.
- La durata in millisecondi restituita da Spotify viene convertita nel formato minuti:secondi.

6.3 Ricerca musicale

La ricerca globale è disponibile anche senza autenticazione. Quando l'utente inserisce almeno due caratteri, l'evento input interroga direttamente Spotify tramite `APIController.searchTrack()`. I risultati vengono mostrati come card con copertina, titolo e artista. Da ogni card è possibile selezionare il brano per inserirlo in una playlist oppure aggiornare il player nel footer.

6.4 Gestione delle playlist private

Le playlist sono memorizzate all'interno dell'oggetto dell'utente. In fase di creazione vengono salvati nome, descrizione, tag, elenco dei brani e copertina. La copertina viene impostata usando l'immagine del primo brano disponibile.

- Creazione di una nuova playlist con nome, descrizione, tag e brani.
- Ricerca dei brani direttamente dal modale di creazione.
- Aggiunta di nuovi brani a una playlist già esistente tramite la ricerca globale.
- Rimozione di singoli brani dalla playlist.
- Modifica del nome di una playlist già esistente.
- Visualizzazione del dettaglio con copertina, descrizione, tag, proprietario, anno e durata dei brani.
- Eliminazione della playlist e rimozione delle eventuali condivisioni collegate.

La modifica di una playlist può avvenire sia sulla sua composizione, tramite aggiunta e rimozione dei brani, sia sul nome. Nel dettaglio della playlist è presente il pulsante "Modifica Nome", che permette al proprietario di inserire un nuovo nome. La modifica viene salvata nel Web Storage e, se la playlist era stata condivisa in una comunità, la copia condivisa viene aggiornata tramite la funzione `saveCurrentUser()`.

6.5 Gestione delle comunità musicali

Gli utenti autenticati possono esplorare le comunità, cercarle per nome o tag, unirsi a una comunità o crearne una nuova. Ogni comunità contiene identificativo, nome, descrizione, tag, creatore e lista dei membri.

- Il creatore può modificare nome, descrizione e tag della comunità.
- Il creatore può eliminare la comunità; l'eliminazione rimuove anche le playlist condivise al suo interno.
- Un membro non creatore può abbandonare la comunità.

- La bacheca mostra le playlist condivise e permette di visualizzare anche gli iscritti con generi e artisti preferiti.

-

All'interno della bacheca è inoltre disponibile una ricerca delle playlist condivise. Il filtro utilizza come criteri i tag associati alla playlist e le canzoni contenute; viene considerato anche il nome del cantante. La ricerca viene effettuata sui dati delle playlist condivise memorizzati nel Web Storage e aggiorna dinamicamente le card visualizzate.

6.6 Condivisione e importazione

Una playlist privata può essere condivisa in una delle comunità di cui l'utente è membro. La condivisione salva nel localStorage una copia della playlist insieme a proprietario e comunità di destinazione. Se il proprietario aggiunge o rimuove brani, `saveCurrentUser()` aggiorna anche la copia presente nella bacheca.

- Condivisione di una playlist in una comunità.
- Revoca della condivisione.
- Visualizzazione della playlist condivisa nella bacheca della comunità.
- Importazione nel proprio profilo da parte di un altro utente.
- Controllo tramite `originalId` per evitare di importare due volte la stessa playlist.

6.7 Player e tema

Il footer contiene un player grafico che aggiorna titolo, artista, copertina e icona play/pausa quando viene selezionato un brano. Il player non effettua streaming audio reale: rappresenta una funzionalità di interfaccia. Il pulsante tema permette invece di passare tra modalità chiara e scura e memorizza la scelta nel browser.

7. Web Storage

La persistenza richiesta dalla traccia è implementata tramite `localStorage`. Poiché Web Storage salva solo stringhe, gli oggetti JavaScript vengono serializzati con `JSON.stringify()` e ricostruiti con `JSON.parse()`.

Chiave	Contenuto
sn4m_users	Array degli utenti registrati. Ogni utente contiene dati personali, preferenze e playlist private.
sn4m_currentUser	Utente autenticato nella sessione corrente.
sn4m_communities	Array delle comunità con dati descrittivi, creatore e membri.
sn4m_shared_playlists	Array delle playlist condivise nelle comunità, con copia della playlist e riferimenti a proprietario e community.
theme-override	Preferenza tema chiaro/scuro.

8. Gestione errori e sicurezza base

- Validazione HTML5 con `required` e `type=email` nei form.
- Controllo duplicati per email, brani già presenti e playlist già importate.
- `try/catch` nelle funzioni che interrogano Spotify e messaggi di errore in interfaccia.
- `confirm()` prima di eliminare playlist, comunità e account.
- `requireLogin()` per impedire l'uso delle funzioni riservate agli utenti non autenticati.
- Aggiornamento coerente del `localStorage` quando vengono modificati utenti, playlist o comunità.

9. Scelte implementative

Scelta	Motivazione
Applicazione a pagina singola	Riduce i ricaricamenti e rende la navigazione più fluida utilizzando sezioni HTML dinamiche.
JavaScript suddiviso per dominio	Separa responsabilità tra utenti, ricerca, Spotify, playlist, comunità e interfaccia generale.
Web Storage	Permette di simulare persistenza senza backend.
Copia della playlist condivisa	Mantiene una rappresentazione autonoma nella community, sincronizzata dal proprietario quando cambiano i brani.
Bootstrap 5.3	Semplifica responsive design e componenti come modali, dropdown e griglie.
Tema chiaro/scuro	Aggiunge una personalizzazione extra e riutilizza le variabili e il sistema di temi Bootstrap.

10. Prove di funzionamento

10.1 Ambiente di testing

- Browser: Google Chrome / Safari.
- Sistema operativo: macOS.
- Esecuzione locale del progetto tramite file HTML e Web Storage del browser.

Le schermate seguenti documentano le principali operazioni previste dalla consegna e mostrano l'effettivo comportamento dell'interfaccia.

10.2 Home e accesso

La Home pubblica presenta il progetto e consente a un utente non autenticato di accedere alla ricerca musicale oppure alla schermata di login/registrazione.

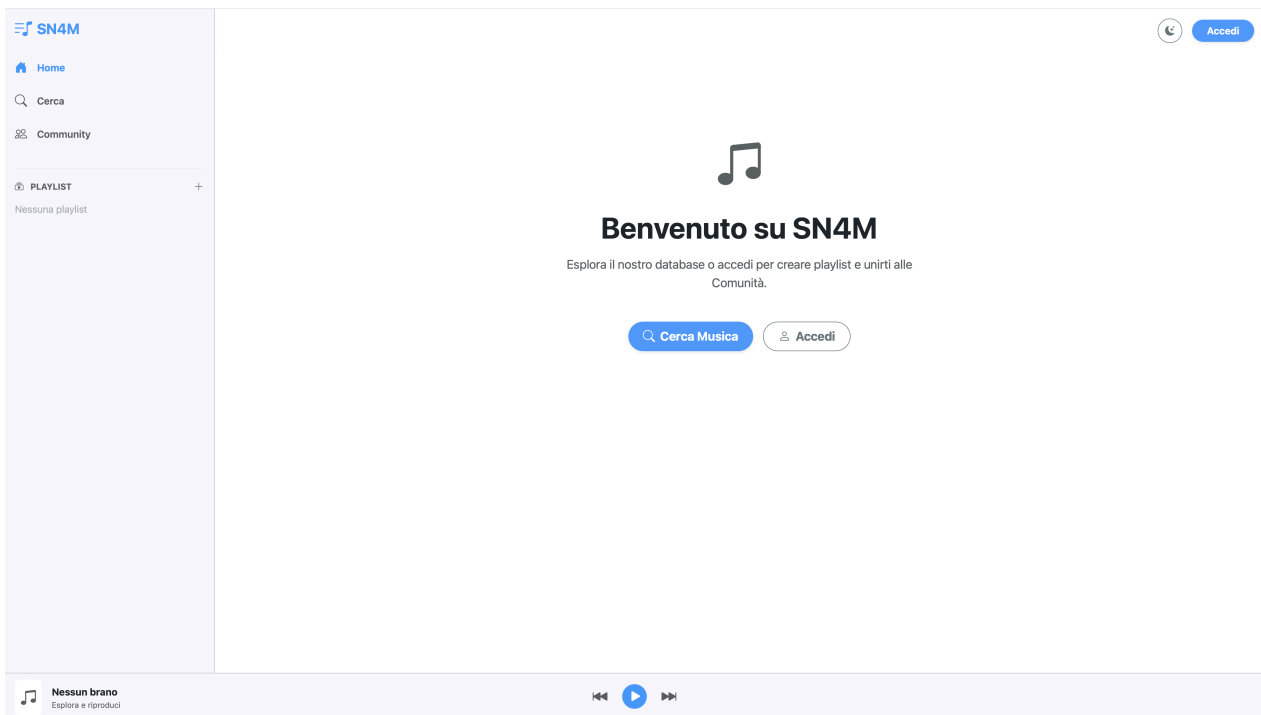


Figura 1 - Home di SN4M visualizzata da un utente non autenticato.

10.3 Registrazione utente

La registrazione raccoglie i dati richiesti dalla traccia: nome utente, email, password, generi musicali preferiti e artisti preferiti.

Crea un Account

Nome Utente

Email

Password

Seleziona Generi Preferiti...

Cerca i tuoi artisti preferiti...

Registrati

Hai già un account? [Accedi](#)

Figura 2 - Form di registrazione con preferenze musicali e ricerca degli artisti.

10.4 Ricerca delle canzoni tramite Spotify

La ricerca globale interroga Spotify e visualizza i risultati sotto forma di card. Nella prova è stata cercata la parola “drake”; i risultati mostrano cover, titolo e artista e permettono di aggiungere un brano a una playlist.

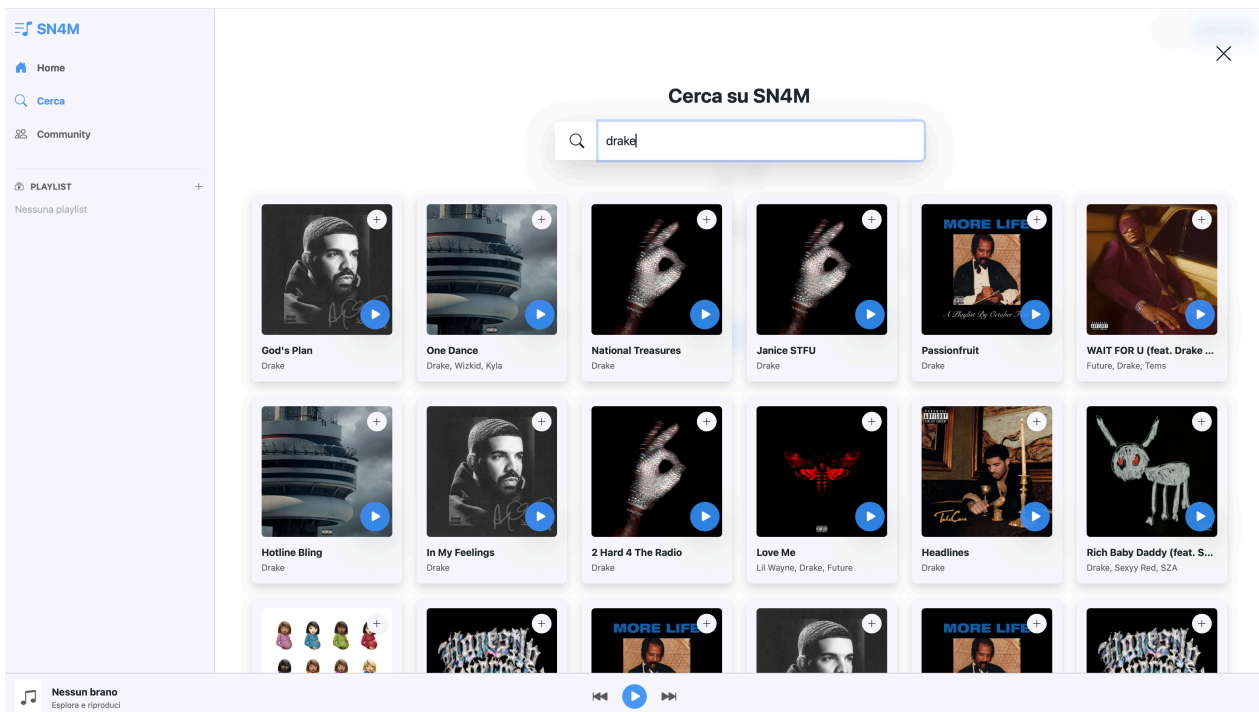


Figura 3 - Ricerca di canzoni tramite Spotify Web API.

10.5 Gestione degli stati vuoti

Quando l'utente non possiede ancora playlist o comunità, l'applicazione mostra messaggi specifici e pulsanti contestuali, evitando aree vuote non spiegate.

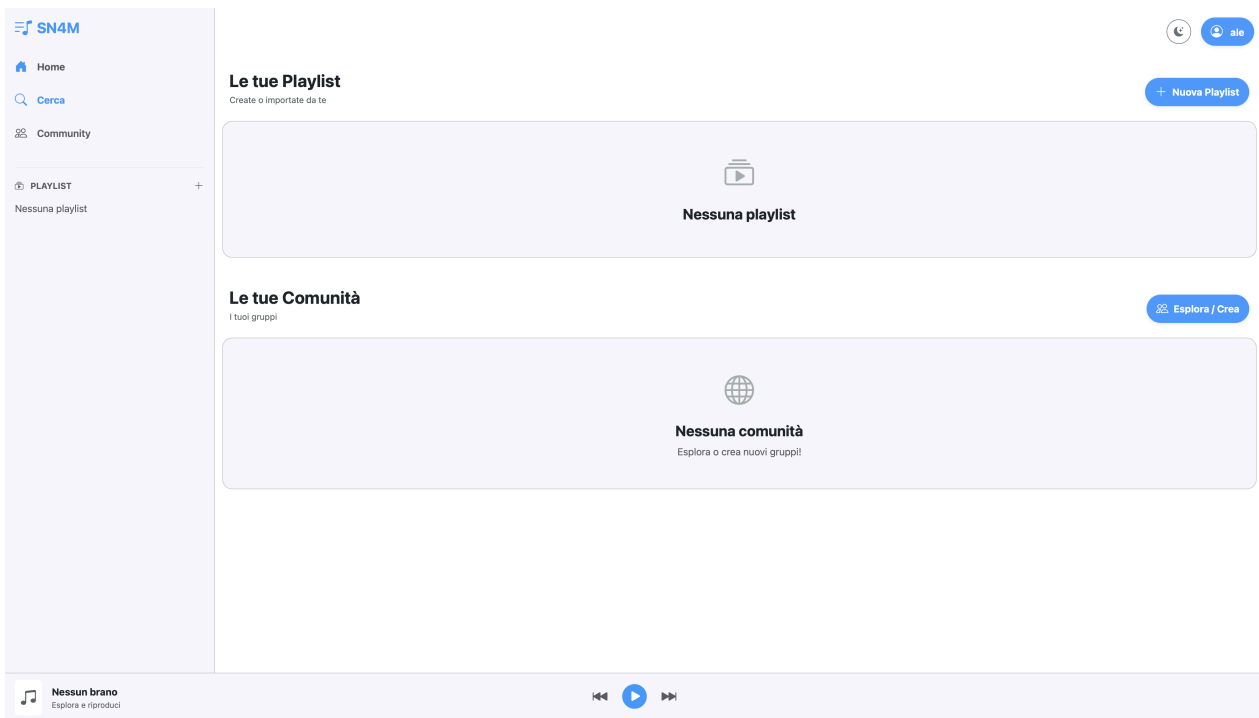


Figura 4 - Stato della Home senza playlist e senza comunità.

10.6 Visualizzazione delle playlist personali

Dopo la creazione, la playlist viene visualizzata nella Home e anche nella sidebar. La card mostra copertina, nome, numero di brani e tag.

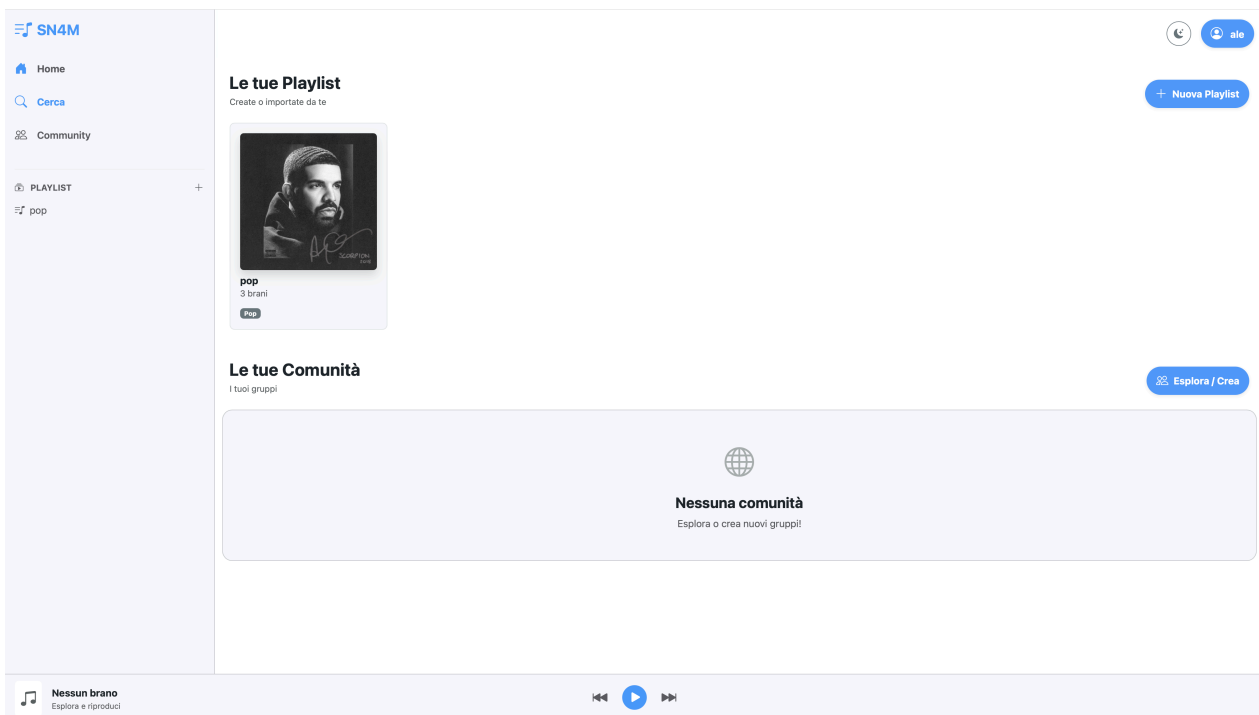


Figura 5 - Home autenticata con la playlist personale "pop".

10.7 Creazione di una playlist

Il modale di creazione permette di inserire nome, descrizione e tag e di cercare brani da aggiungere prima del salvataggio.

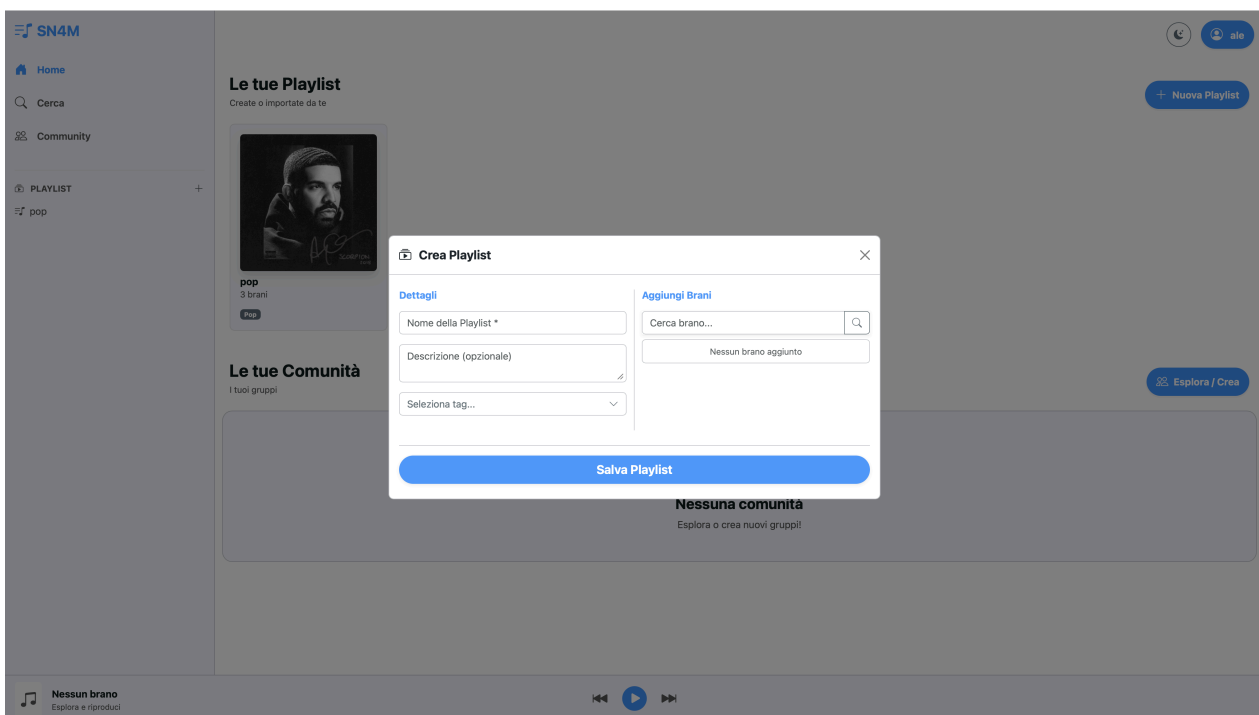


Figura 6 - Modale per la creazione di una nuova playlist.

10.8 Dettaglio e gestione della playlist

Il dettaglio della playlist mostra i brani in una tabella con titolo, artista, anno e durata. Sono presenti le azioni per aggiungere brani, modificare il nome della playlist, condividerla in una comunità, eliminare singole canzoni ed eliminare l'intera playlist.

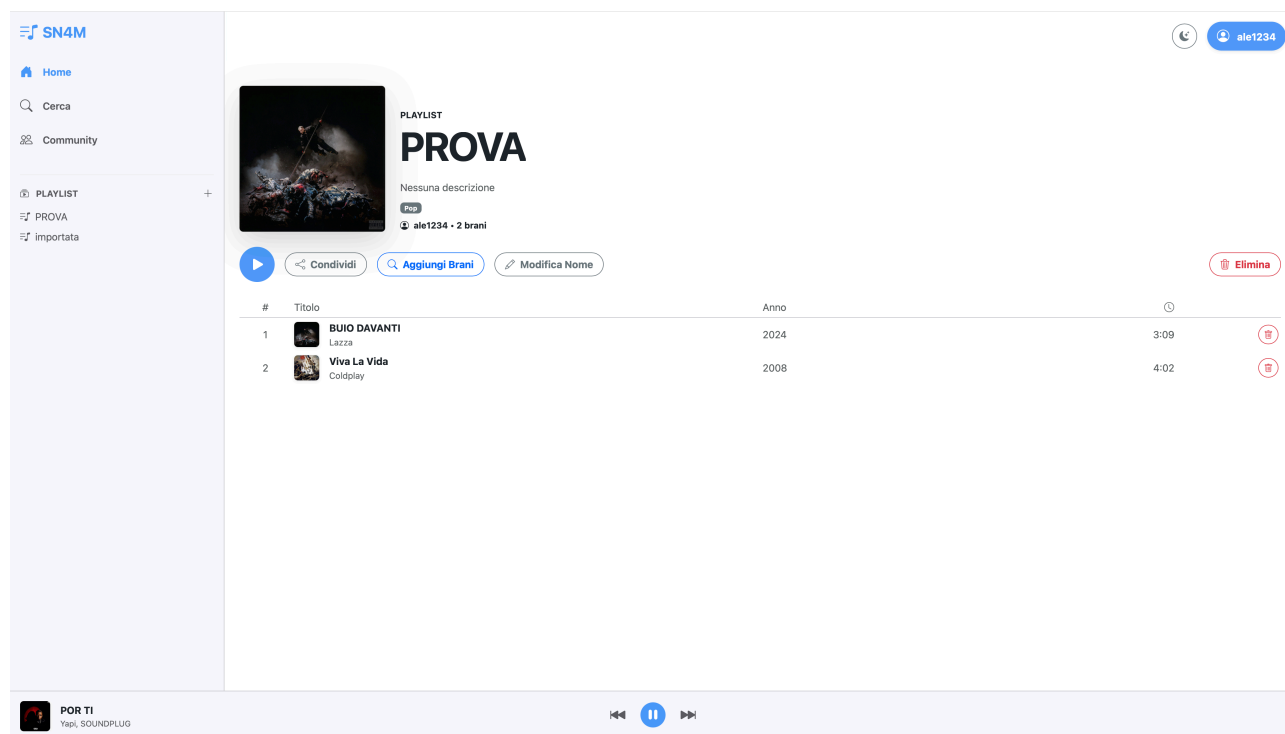


Figura 7 - Dettaglio della playlist con elenco dei brani e azioni disponibili.

10.9 Gestione del profilo

Il profilo consente di modificare username, email, generi e artisti preferiti. Nello stesso modale sono disponibili logout ed eliminazione definitiva dell'account.

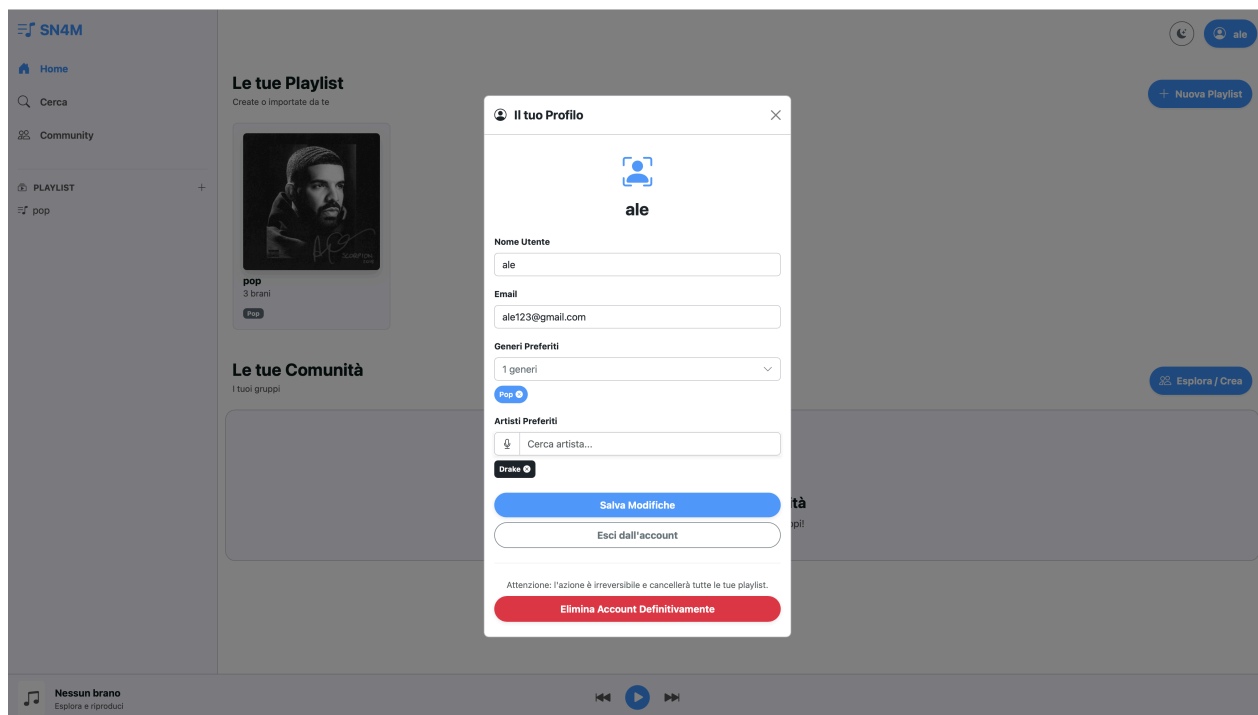


Figura 8 - Modale di visualizzazione e modifica del profilo utente.

10.10 Ricerca e gestione delle comunità

Il modale “Gestione Comunità” presenta due sezioni: Esplora/Le tue e Crea Nuova. La ricerca permette di trovare comunità esistenti e, quando non sono presenti risultati, viene mostrato un messaggio dedicato.

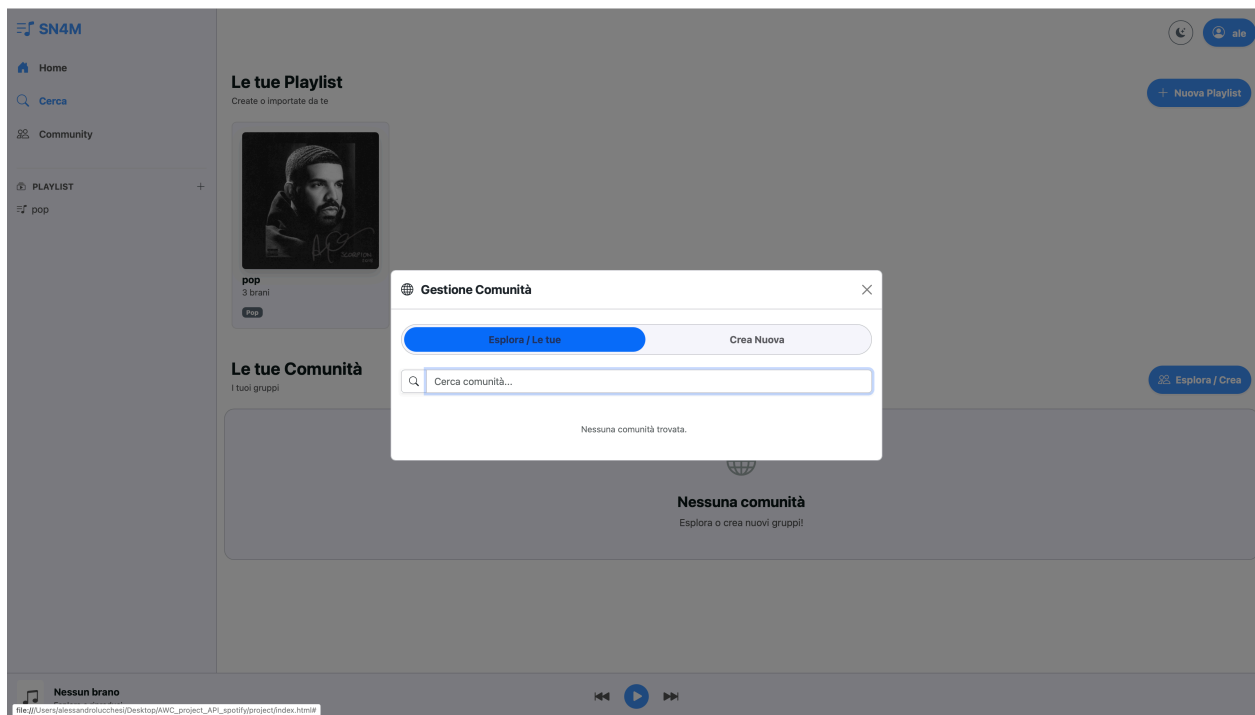


Figura 9 - Ricerca comunità senza risultati.

Quando viene trovata una comunità, l'utente può visualizzare il numero di membri e utilizzare il pulsante “Unisciti” per entrarvi.

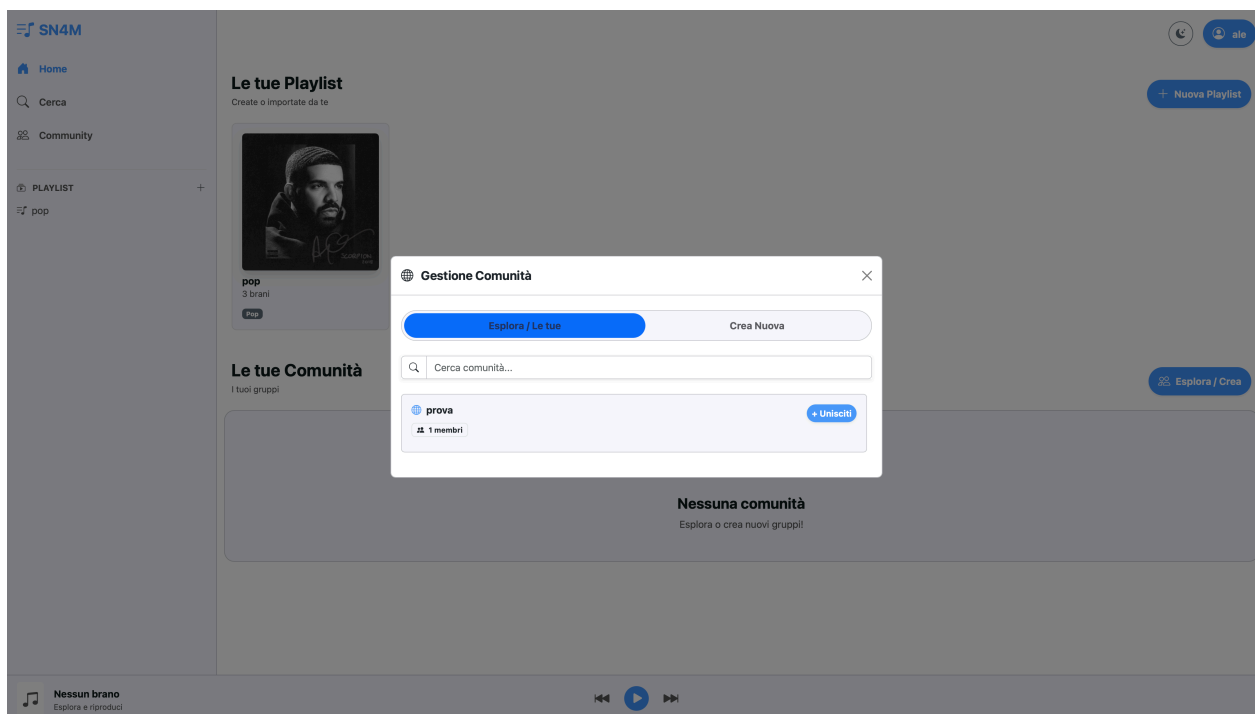


Figura 10 - Comunità trovata e pulsante per unirsi al gruppo.

10.11 Condivisione di una playlist in una comunità

Dal dettaglio di una playlist privata l'utente può aprire il modale di condivisione e scegliere una delle comunità di cui è membro. La playlist viene quindi resa disponibile nella bacheca della comunità.

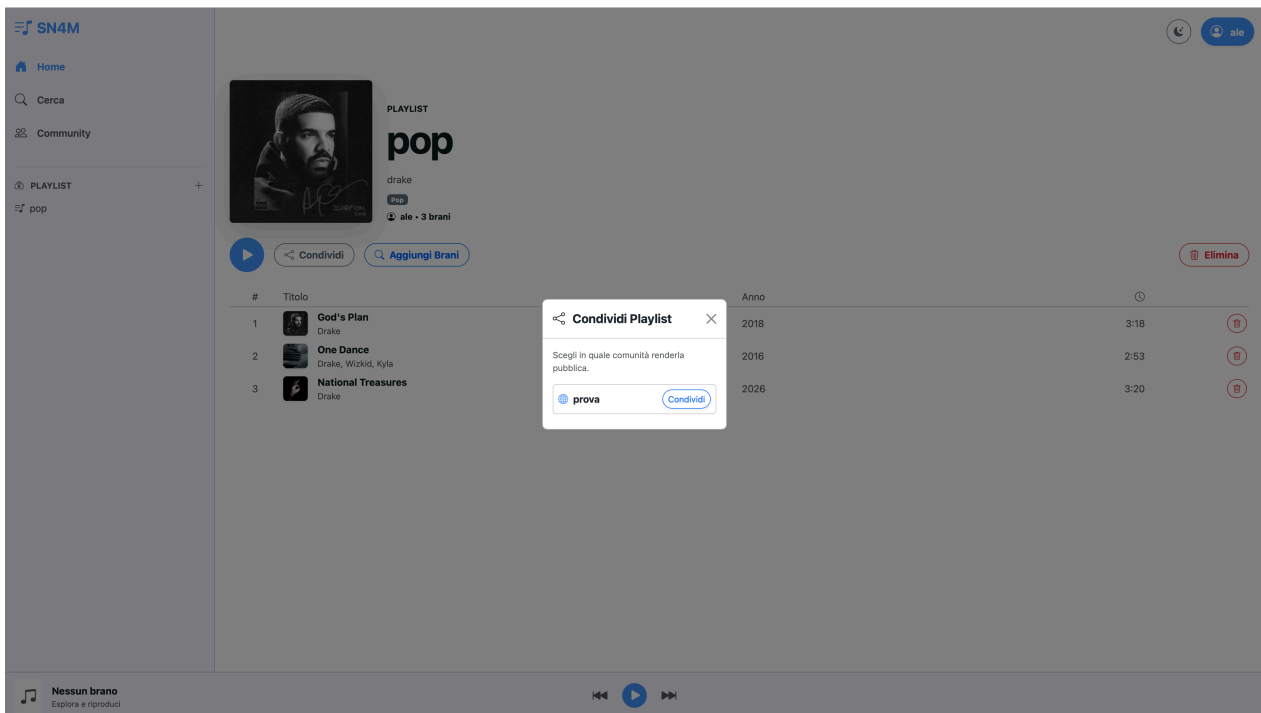


Figura 11 - Selezione della comunità in cui condividere la playlist.

10.12 Ricerca delle playlist condivise

Nella bacheca di una comunità l'utente può cercare tra le playlist condivise utilizzando il campo di ricerca dedicato. Il filtro considera i tag della playlist e le canzoni contenute, verificando sia il titolo del brano sia il cantante. In questo modo è possibile individuare rapidamente le playlist di interesse tra quelle condivise dagli utenti della comunità.

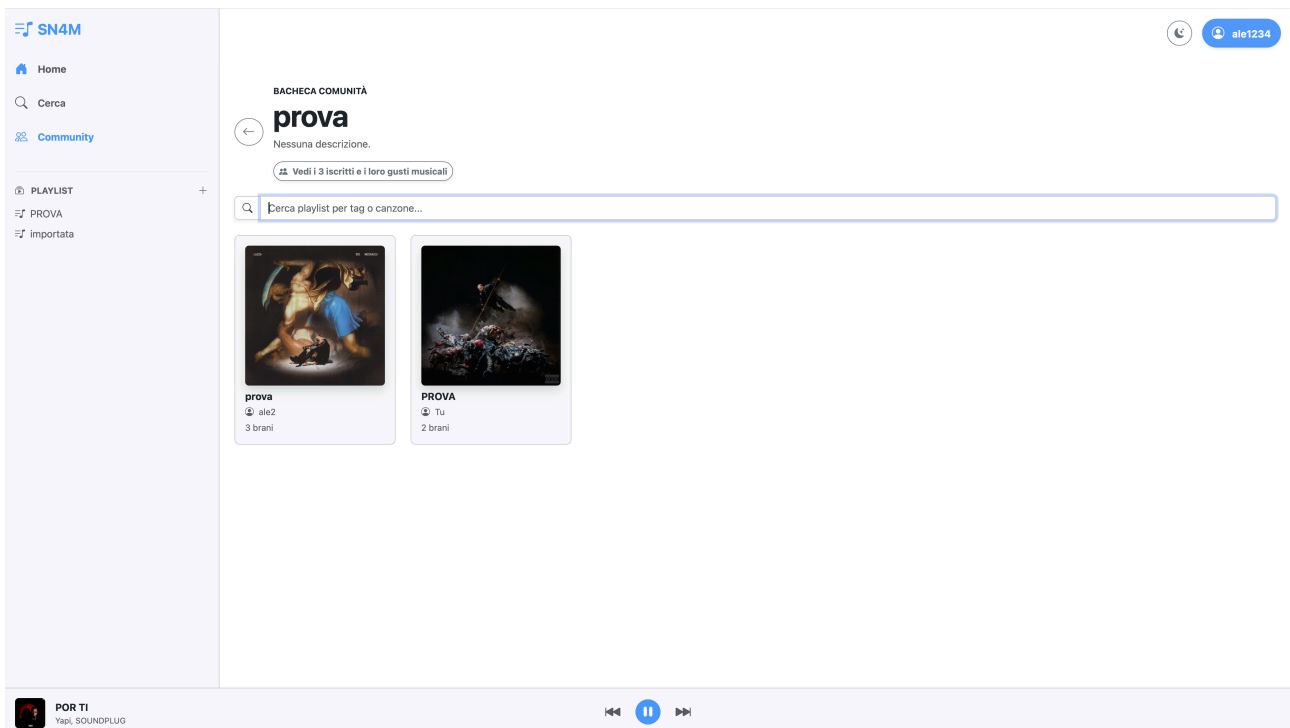


Figura 12 - Ricerca di una playlist condivisa nella bacheca della comunità.

11. Corrispondenza con le operazioni richieste

Operazione	Realizzazione
Registrazione e login	Implementati in user.js tramite handleRegister() e handleLogin().
Gestione dati utente	Modifica profilo, logout ed eliminazione account.
Playlist private	Creazione, modifica del nome e del contenuto tramite aggiunta/rimozione brani, visualizzazione ed eliminazione.
Comunità musicali	Creazione, ricerca, iscrizione, modifica, abbandono ed eliminazione.
Playlist condivise	Condivisione, sincronizzazione della copia, revoca, visualizzazione in bacheca e ricerca per tag e canzoni contenute.
Visualizzazione canzoni	Tabella di dettaglio playlist e card nei risultati di ricerca.
Ricerca canzoni	Ricerca Spotify per query testuale con risultati live.
Playlist private e condivise	Home per le private; bacheca community per le condivise.
Importazione playlist condivisa	Implementata in community.js con importCurrentSharedPlaylist().
Ricerca playlist condivise	Filtro nella bacheca della comunità per tag, titolo delle canzoni e cantante tramite filterSharedPlaylists().

12. Conclusioni

SN4M realizza una piattaforma client-side per la gestione di musica, playlist e comunità, integrando i dati dinamici di Spotify con la persistenza locale richiesta dal progetto. La separazione tra HTML, CSS e moduli JavaScript rende la struttura del progetto leggibile e facilmente modificabile. L'applicazione copre i principali scenari previsti dalla consegna Full e aggiunge funzionalità di interfaccia quali tema chiaro/scuro, ricerca live tramite Spotify Web API e player grafico.

Le scelte implementative sono state orientate alla semplicità, alla modularità e alla coerenza con i requisiti del corso, mantenendo tutta la logica nel browser e utilizzando il Web Storage come sorgente persistente delle informazioni.

Riferimenti

- Specifica del progetto “Social Network for Music (SN4M)” - Applicazioni Web e Cloud, A.A. 2025/2026.
- Spotify Web API - utilizzata per autenticazione applicativa, categorie, ricerca brani e ricerca artisti.
- Bootstrap 5.3 e Bootstrap Icons - utilizzati per layout, componenti responsive e icone.